Partie 1 : Technologie React

Semaine 1 — Presentation de React
Environnement de développement

Semaine 2 — Développement avec JSX
Création des Composants

Semaine 3 — Configuration des composants : Props, Etat Local

Semaine 4 — Interaction avec Utilisateur : Evenements, Formulaires

Semaine 5 — Communiquer avec serveur
Routage

Semaine 6 — Communication inter-composants

React

Interaction avec Utilisateur :

Évènements, Formulaires

UNIVERSIAPOLIS
Université Internationale d'Agadir
الجامعة الدولية لأكادير
ECOLE POLYTECHNIQUE D'AGADIR

Gestion des événements

- **Événements React :**
  - Tout comme HTML, React peut effectuer des actions en fonction des événements utilisateur.
  - React a les mêmes événements que HTML: **onClick**, **onChange**, **onMouseOver**, etc.

Exemple :

```
<button onClick={this.handleClick}>Cliquer</button>

handleClick(){
  alert("Bonjour à toutes et à tous...")  }
```

NB : En React, l'appel de la fonction se fait plutôt comme suit : **{this.handleClick}**

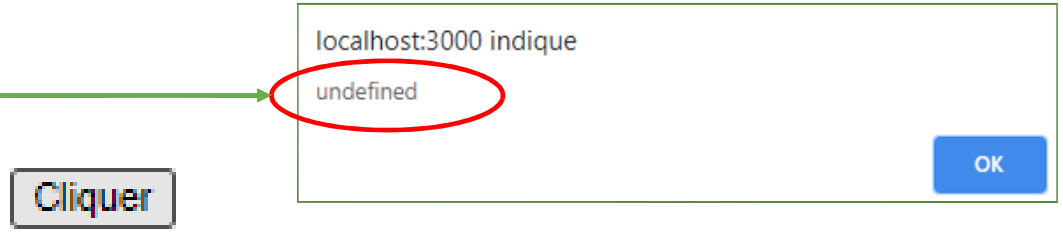
Et non pas : ~~{this.handleClick()}~~

Gestion des événements

- **Binding :**
  - Pour les méthodes dans React, le mot clé **this** doit représenter le composant propriétaire de la méthode
  - On devrait préciser à une fonction quelle valeur utiliser pour la variable **this**.

```
<button onClick={this.handleClick}>Cliquer</button>

handleClick(){
  alert(this)
}
```



Gestion des événements

- **Solutions :**
  - 1ère solution : utiliser **bind(this)**

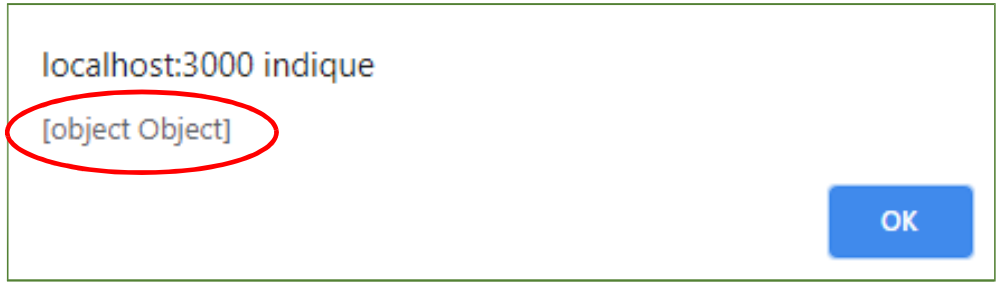
```
<button  onClick={this.handleClick.bind(this)}>Cliquer</button>
```

  - 2ème solution : utiliser une fonction fléchée pour garder la même valeur de this

```
<button  onClick={()  =>  this.handleClick()}>Cliquer</button>
```

  - 3ème solution : déclarer la fonction comme une fonction fléchée :

```
handleClick  =  ()  =>  {
alert(this)
}
```



Gestion des événements

- **Passage des arguments :**
  - A part **this**, il y a aussi la possibilité de passer d'autres arguments au niveau de l'appel de la fonction comme suit : **bind(this, argument)**

```
<button  onClick={this.handleClick.bind(this,"bonjour")}>Cliquer</button>

handleClick  (msg)  {
   alert(msg)
}
```

Remarque :

- Si on envoie les arguments sans utiliser la méthode bind, la fonction sera exécutée lorsque la page sera chargée sans attendre que le bouton soit cliqué.

Gestion des formulaires

- **Composants de formulaires en React :**
  - Tout comme en HTML, React utilise des formulaires pour permettre aux utilisateurs d'interagir avec la page Web.
  - Au niveau des formulaires en React, on utilise simplement la ***prop value=***, quel que soit le type du composant.
  - Quand il s'agit de valeurs multiples, comme par exemple pour ***<select multiple>*** et ses enfants ***<option>***, on pourrait faire recours à un tableau de valeurs à props.

Gestion des formulaires

- **Composants de formulaires en React :**
  - React utilise le concept de « composants contrôlés » pour les formulaires.
    - Accès à la valeur du champ : ***event.target.value***

```
this.state = {
        . . . .
        personne:''
};

<form>
  <h2>Bonjour {this.state.personne}</h2>
  <input type="text" onChange={this.changeHandler}/>
  <button>Ajouter personne</button>
</form>


changeHandler = (event) => {
    this.setState({personne : event.target.value});
}
```

Gestion des formulaires

- **Soumission d'un formulaire : onSubmit**
  - Contrôler l'action de soumission en ajoutant un gestionnaire d'événements dans l'attribut **onSubmit**.

```
<form onSubmit={this.addPerson}>
  <h2>Bonjour {this.state.personne}</h2>
  <input type="text" onChange={this.changeHandler}/>
  <button>Ajouter personne</button>
</form>
```

```
addPerson = (event) => {
  event.preventDefault();
  alert("" + this.state.personne);
}
```

event.preventDefault : Empêcher le rechargement / rafraîchissement du navigateur

Gestion des formulaires

- **Ajout d'un élément à une liste :**
  - Contrôler l'action de soumission en ajoutant un gestionnaire d'événements dans l'attribut ***onSubmit***.

```
this.state = {
  monTexte: "Bienvenue à cette formation React",
  personnes:["Ali", "Salah", "Mohamed"],
  personne:''
};
```

```
addPerson= (event) => {
  event.preventDefault();
  let tab_personnes= this.state.personnes.slice();
  tab_personnes.push(this.state.personne);
  this.setState({
    personnes:tab_personnes
  })
}
```

```
this.setState({
  personnes:[...this.state.personnes,this.state.personne]
})
```

(Autre façon de faire)

slice : Créer une copie du tableau

Gestion des formulaires

- **TextArea :**
  - Les input ***TextArea*** utilisent plutôt un attribut value

```
render()
{
    return (
        <form onSubmit={this.handleSubmit}>
            <label>
                Description :
                <textarea value={this.state.value} onChange={this.handleChange}/>
            </label>
            <input type="submit" value="Submit"/>
        </form>
    );
}
```

Gestion des formulaires

- **Selected :**
  - Pour être conforme avec les autres types de formulaires, React utilise l'attribut value au lieu selected pour déterminer l'élément sélectionné.

```
render() {
    return (
        <form onSubmit={this.handleSubmit}>
            <label>
                Choisissez votre framework préféré:
                <select value={this.state.value} onChange={this.handleChange}>
                    <option value= "Angular">Angular</option>
                    <option value= "React">React</option>
                    <option value= "Vue">Vue.Js</option>
                </select>
            </label>
            <input type="submit" value="Submit" />
        </form>
    );
}
```

Application

Formulaire de presentation

Prénom :

fadwa

Nom :

lachhab

E-mail :

fadwa.lachhab@e-polytechnique.ma

Envoyer

Résumé des informations :

Prénom : fadwa, Nom : lachhab, E-mail : fadwa.lachhab@e-polytechnique.ma